

Filière : Master IESE – S2

Travaux Pratiques : Robotique Industrielle

TP 2: Modélisation et Commande des Robots Manipulateurs

Objectifs

Ce TP a pour objectifs :

- Appliquer les notions de MGD et MDD dans des cas réels
- Modéliser et commander un robot SCARA à 3 DDL en utilisant MATLAB/SIMULINK

Robot SCARA

1- Description

On s'intéresse à un robot SCARA à 3 DDL (RRP). Le schéma du robot est représenté sur la figure 1, qui montre les différents degrés de liberté, ainsi que les paramètres géométriques du robot. Un exemple de robot réel de cette classe est représenté sur la figure 2, il s'agit d'un robot de la société ROSIER-SOCITEC.

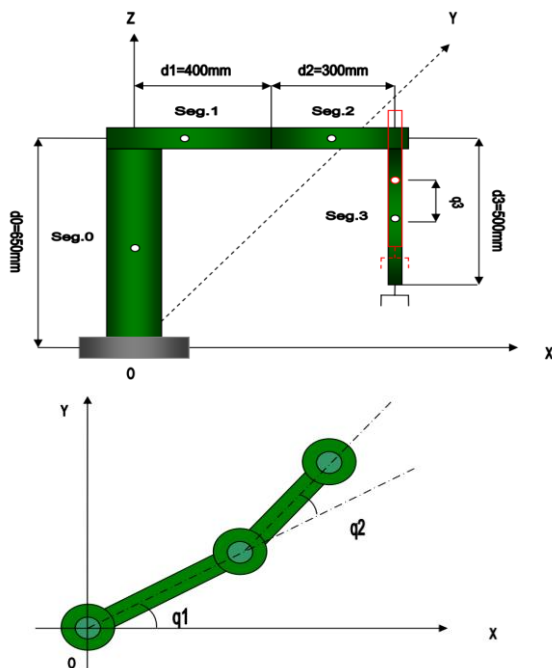


Figure 1 : Schéma du robot SCARA à 3 DDL.



Figure 2 : Exemple de robot SCARA à 3 DDL (ROSIER-SOCITEC)

2- Modèle dynamique du robot

Le modèle dynamique Lagrangien du robot est donné par l'équation différentielle suivante :

$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$; ou $q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \end{pmatrix}$, $\dot{q} = \begin{pmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{pmatrix}$, $\ddot{q} = \begin{pmatrix} \ddot{q}_1 \\ \ddot{q}_2 \\ \ddot{q}_3 \end{pmatrix}$ sont les vecteurs de position, de vitesse et d'accélération.

$M(q) \in \mathcal{R}^{3 \times 3}$ est la matrice d'inertie (symétrie définie positive)

$C(q, \dot{q}) \in \mathcal{R}^{3 \times 3}$ est la matrice des effets de Coriolis et centrifuges

$G(q) \in \mathcal{R}^3$ est le vecteur de gravité

$\tau \in \mathcal{R}^3$ est le vecteur des couples

Les différentes matrices de ce modèle dynamique sont détaillées en annexe. Les paramètres géométriques et dynamiques du robot sont regroupés dans le tableau 1.

Table 1 : Paramètres dynamiques du robot SCARA

Segement	Segement 0	Segement 1	Segement 2	Segement 3
Masse [Kg]	$m_0 = 19.5$	$m_1 = 8$	$m_2 = 6$	$m_3 = 0.5$
Longueur [m]	$d_0 = 0.65$	$d_1 = 0.4$	$d_2 = 0.3$	$d_3 = 0.3$
Inertie [$Kg m^2$]	$I_0 = 1.0298$	$I_1 = 0.16$	$I_2 = 0.0675$	$I_3 = 0.0056$

Travail demandé

1- Configurations générales

- ➔ Ouvrir Matlab/Simulink ➔ Créer un nouveau modèle Simulink ➔ Sauvegarder le modèle sous le nom Robot_SCARA
- ➔ Accédez aux paramètres du modèle (Model Settings), puis :
 - Sélectionnez un pas d'échantillonnage fixe (Fixed-step size) de 0,004s.
 - Réglez le solveur (Solver) sur Automatique (Auto).

2- Modélisation du robot SCARA

- 2-1 Proposer une fonction MATLAB permettant la détermination du modèle géométrique direct d'un robot quelconque connaissant les différentes paramètres DH. Tester cette fonction sur le cas du robot SCARA étudié.
- 2-2 Dans votre modèle Simulink, implémentez le modèle dynamique du robot SCARA (voir équations en annexe) en utilisant un bloc « MATLAB Function »
- 2-3 Simuler le robot SCARA en boucle ouverte (commande nulle). Avec $t_f = 2 s$ est la durée de simulation (stop time = 2) et $q_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$, $\dot{q}_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$. Tracer les courbes d'évolution des coordonnées articulaires du robot (positions et vitesses). Utilisez des

Dans le but de réaliser un suivi des trajectoires de référence ainsi générées, on se propose d'explorer l'ensemble de commande représenté sur la figure 4.

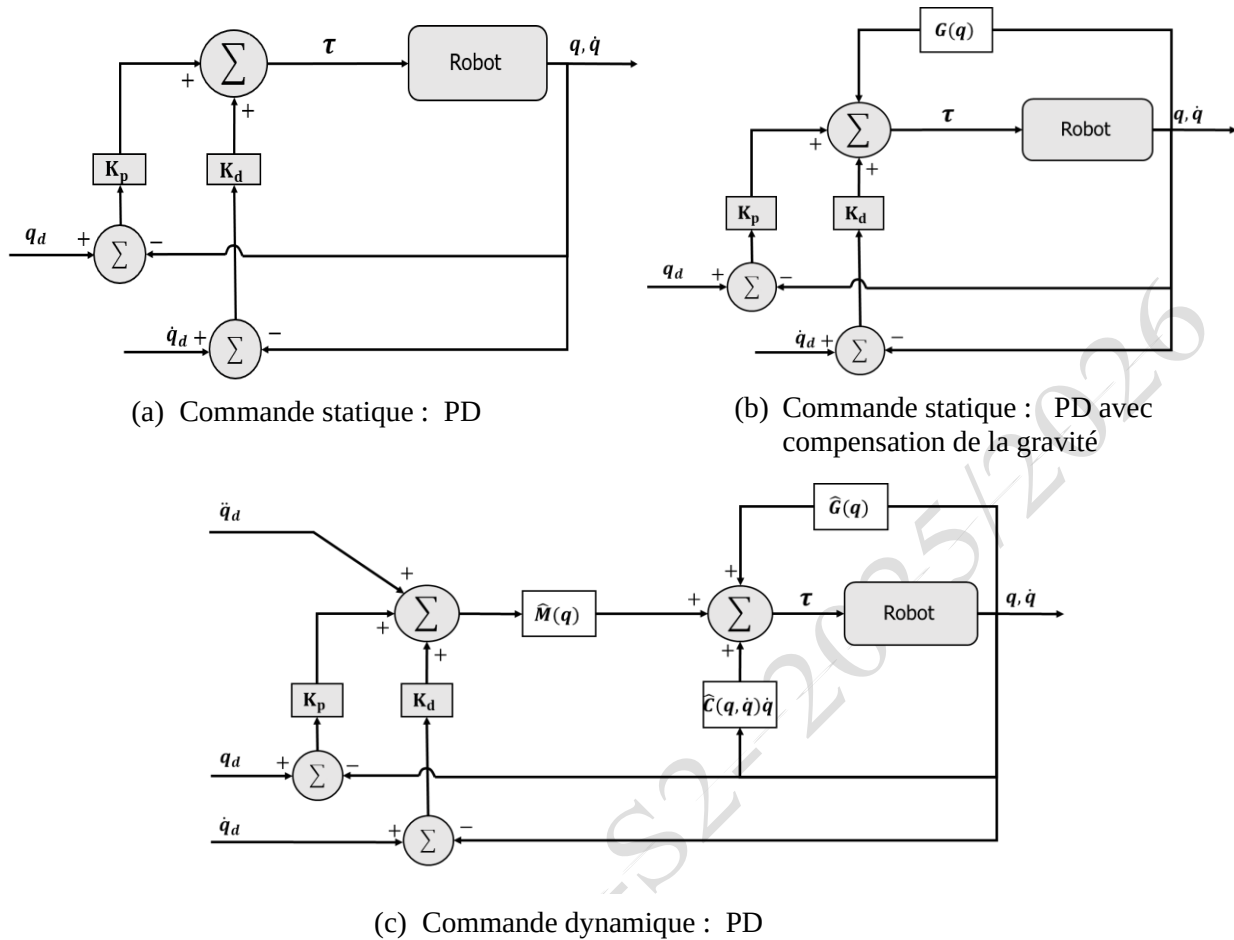


Figure 4 : Schéma des différentes stratégies de contrôle

- 3-4 Coder les trois méthodes de contrôle sous Simulink (chaque méthode dans un sous-système), puis utiliser un « Multi-Port Switch » pour basculer entre les méthodes de contrôle lors de la simulation.

$$\begin{cases} K_{p1} = K_{p2} = K_{p3} = 450 \\ K_{d1} = K_{d2} = K_{d3} = 35 \end{cases}$$

Sauvegardez les résultats de simulation à l'aide du bloc « To Workspace ». Tracez les résultats en utilisant la fonction subplot afin de diviser chaque figure en trois sous-figures. Sur chaque sous-figure, tracez à la fois la courbe de la consigne et les réponses des trois contrôleurs.

- 3-5 Pour des entrées q_d de type échelon, comparez les performances (temps de réponse, dépassement et erreur statique) des trois méthodes. Tracez les réponses en utilisant la fonction subplot, puis utilisez la fonction stepinfo pour déterminer les indicateurs de performance et groupez ces résultats dans un tableau.

$$\text{Avec } q_d = \begin{pmatrix} q_{1d} = 0,2 \text{ rad} \\ q_{2d} = 0,2 \text{ rad} \\ q_{3d} = 0,1 \text{ m} \end{pmatrix}, q_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \dot{q}_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, t_f = 3 \text{ s}$$

Remarque : Organisez votre modèle en utilisant les blocs « Goto » et « From » pour

remplacer les connexions directes entre les sous-systèmes.

4 Commande en espace opérationnel

La commande dans l'espace opérationnel a pour objectif d'asservir la position de l'OT X à la trajectoire X_d préprogrammée. Le schéma synoptique de cette commande est représenté sur la figure 5.

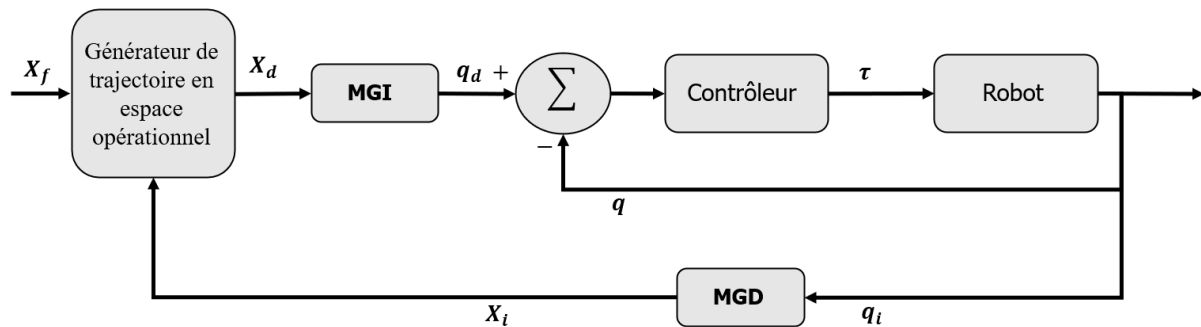


Figure 5 : Structure de la commande en espace opérationnel

A- Génération de trajectoire et commande statique/dynamique

On souhaite générer des trajectoires de mouvement dans l'espace cartésien permettant au robot d'aller d'une position initiale P_i (à vitesse nulle) à une position finale P_f (à vitesse nulle), exprimées par :

$$P_i = \begin{pmatrix} x_i \\ y_i \\ z_i \end{pmatrix} = \begin{pmatrix} 0.4 \\ 0 \\ 0.35 \end{pmatrix} ; P_f = \begin{pmatrix} x_f \\ y_f \\ z_f \end{pmatrix} = \begin{pmatrix} 0.6 \\ 0.2 \\ 0.3 \end{pmatrix}$$

- 4-1 Par utilisation de la méthode de génération de trajectoire point à point (Interpolation de degré 5), construire une fonction Matlab « Matlab Function » qui permet de calculer les trajectoires de référence (de positions, de vitesses et d'accélération cartésiennes) à l'instant t . Cette fonction aura comme entrée (t, P_i, P_f, t_f) et comme sorties. Tester cette fonction sur une durée de 3 s.

$$X_d = \begin{pmatrix} x_d \\ y_d \\ z_d \end{pmatrix}, \dot{X}_d = \begin{pmatrix} \dot{x}_d \\ \dot{y}_d \\ \dot{z}_d \end{pmatrix}, \ddot{X}_d = \begin{pmatrix} \ddot{x}_d \\ \ddot{y}_d \\ \ddot{z}_d \end{pmatrix}$$

- 4-2 Comparer la réponse des trois méthodes de contrôle

Remarques :

⇒ Un seul fichier est à rendre, format .zip, contenant le rapport et les fichiers de simulation.

4-4 Modèles géométriques direct et inverse

Le modèle géométrique direct (**MGD**) du robot exprime la situation de l'organe terminal du robot dans l'espace opérationnel en fonction de ses coordonnées articulaires. Il est donné par l'équation suivante

$$X = f(q)$$

Où X est le vecteur de position et d'orientation de l'organe terminal, q est le vecteur des coordonnées articulaires du robot. Dans le cas du robot SCARA à 3ddl on considère.

$$q = (q_1, q_2, q_3)^T ; X = (P_x, P_y, P_z)^T$$

Où P_x, P_y, P_z correspondent aux coordonnées cartésiennes de l'organe terminal. Le modèle géométrique direct est donné par le code Matlab suivant :

```
%----- MGD -----
Px=d1*cos(q1)+d2*cos(q1+q2); %
Py=d1*sin(q1)+d2*sin(q1+q2); %
Pz=d0-(d3-q3); %
%-----
```

Le modèle géométrique inverse (**MGI**), quant à lui, consiste à trouver les coordonnées articulaires qui amènent l'organe terminale dans une situation désirée, spécifiée par ses coordonnées opérationnelles. Dans le cas du robot SCARA à 3ddl, connaissant P_x, P_y, P_z , le but est de trouver q_1, q_2, q_3 en fonction de P_x, P_y, P_z . Le modèle obtenu est donné par le code suivant :

```
%----- MGI ----- spécifiée
q2 = acos((Px^2+Py^2-d1^2-d2^2)/(2*d1*d2));
B1 = d1+d2*cos(q2);
B2 = d2*sin(q2);
q1 = asin((Py*B1-Px*B2)/(B1^2+B2^2));
q3 = Pz-d0+d3;
%-----
```